



# FOREIGN EXCHANGE REVENUE & RISK ANALYSIS



End to end data project | CSV → MySQL → SQL → Python → Excel → Tableau

CSV FILES

MYSQL DATABASE

SQL AUDIT

PYTHON CLEANING

EXCEL REPORTS

TABLEAU DASHBOARD

This project turns raw international payment data into a live FX revenue & risk dashboard. Three csv files move into a MySQL database, get audited with SQL, cleaned in Python, exported as Excel reports, and finish as a Tableau dashboard.

## STEP 1 LOADING RAW DATA INTO MYSQL

(CODE PREVIEW) data\_transfer2.py

```
files = ['CSV/fx_rates.csv', 'CSV/international_payments.csv',
        'CSV/payment_status.csv']

for a in files:
    tables = a.replace('CSV/', '').split('.')[0]
    df = pd.read_csv(a)
    df.to_sql(tables, engine, if_exists='replace')
    print(# confirms each table loaded)
```

### What this does

Each raw csv gets pushed into its own MySQL table using SQLAlchemy + pandas, so the project has a real queryable database instead of three loose files. The load runs inside a try/except so a bad file fails loudly instead of silently.

### Source Files Loaded

| File                       | Rows  | Columns                                                                | Becomes Table          |
|----------------------------|-------|------------------------------------------------------------------------|------------------------|
| fx_rates.csv               | 3,047 | rate_id, date, currency_pair, exchange_rate                            | fx_rates               |
| international_payments.csv | 4,500 | payment_id, rate_id, sender/receiver country, amount_sent, fee_charged | international_payments |
| payment_status.csv         | 4,387 | payment_id, status, delivery_time_hours, failure_reason                | payment_status         |

## STEP 2 AUDITING THE DATA WITH SQL

Before any cleaning, plain SQL checks were run across all three tables to find nulls, impossible negative values, messy text, and duplicates — every issue found here became a task fixed in Python.

### (CODE PREVIEW) SQL\_Analytics2.sql — audit checks

```
-- missing values across each table
select sum(amount_sent is null) as null_amount,
       sum(fee_charged is null) as null_fee
from international_payments;

-- impossible negative values
select exchange_rate from fx_rates
where exchange_rate < 0;

-- duplicate rows removed before joining
create table fx_rates_clean
select distinct * from fx_rates;
```

### (CODE PREVIEW) SQL\_Analytics2.sql — clean views

```
-- view 1: payments joined to their fx rate
create view v_payment_rates as
select i.payment_id, i.amount_sent, i.fee_charged,
       f.currency_pair, f.exchange_rate
from international_payments_clean i
left join fx_rates_clean f
  on i.rate_id = f.rate_id;

-- view 2: payments joined to delivery status
create view v_delivery_status as ...
```

### Issues The Checks Surfaced

- **fx\_rates** — currency pairs written inconsistently (dashes, underscores, mixed case)
- **international\_payments** — sender/receiver country casing inconsistent; some negative amounts and fees; fee\_charged stored with a "\$" symbol as text
- **payment\_status** — status text inconsistently cased; failed payments sometimes missing a failure\_reason
- Duplicate rows existed in all three raw tables — removed with `create table ... select distinct * from` before any joins were built

### Why Two Views

Rather than re-joining tables inside Python, two clean SQL views (**v\_payment\_rates** and **v\_delivery\_status**) were built on top of the de-duplicated tables. Python simply reads these two views straight in — the join logic lives in one place, not scattered across scripts.

## STEP 3 CLEANING EVERYTHING IN PYTHON

The two SQL views load straight into pandas, then every issue the audit found gets fixed column by column before anything is reported on.

### (CODE PREVIEW) python\_analytics2.py — fixing columns

```
# country casing
df_payment['sender_country'] = df_payment['sender_country']\
    .str.strip().str.upper()

# strip "$" then force numeric + positive
df_payment['fee_charged'] = df_payment['fee_charged']\
    .str.replace('$', '')
df_payment['fee_charged'] = pd.to_numeric(
    df_payment['fee_charged'], errors='coerce').abs()

# normalise currency pair symbols
df_payment['currency_pair'] = df_payment['currency_pair']\
    .str.replace('-', '/').str.replace('_', '/')
```

### (CODE PREVIEW) python\_analytics2.py — flagging records

```
# merge payments with delivery status
merged_df = pd.merge(df_payment, df_delivery,
                    on='payment_id', how='left')

# classify every incomplete transaction
conditions = [
    merged_df['amount_sent'].isna(),
    merged_df['exchange_rate'].isna(),
    (merged_df['status']=='Completed') &
    merged_df['delivery_time_hours'].isna()]
label = ['Missing Financials', 'Missing Rate & Pair',
        'Missing Delivery Time']
merged_df['audit_status'] = np.select(
    conditions, label, default='Passed')
```

### What This Does

- Sender and receiver countries standardized to uppercase; amounts, fees and exchange rates forced to numeric and made positive with **.abs()**
- Currency pairs unified into one **USD/GBP** style format regardless of how they were separated in the raw data
- Delivery status text standardized; failed payments missing a reason get labeled **"No Record"** instead of being left blank
- Payments and delivery status are merged on **payment\_id**, then every row is classified with **np.select()** into one of four failure reasons — or **Passed** if the record is complete

### Why It Matters

This single **audit\_status** column is what both final reports are built from — failed rows drive the Operational Failure report, and only "Passed" rows are trusted enough to feed the FX Corridor revenue report.

## STEP 4 BUILDING THE TWO FINAL REPORTS

### (CODE PREVIEW) Operational Failure report

```
failed = merged_df[merged_df['audit_status']!='Passed']
Operational_Failure = failed.groupby('audit_status').agg(
    Transaction_Volume=('payment_id', 'count'),
    Total_Impacted_Amount=('amount_sent', 'sum'),
    Fee_at_risk=('fee_charged', 'sum'))
Operational_Failure['Financial_Impact'] = (
    Total_Impacted_Amount / total_bank_amount *100)
```

### (CODE PREVIEW) FX Corridor report

```
passed = merged_df[merged_df['audit_status']=='Passed']
FX_Corridor = passed.groupby('currency_pair').agg(
    Total_Fee_Revenue=('fee_charged', 'sum'),
    Total_Amount_Sent=('amount_sent', 'sum'),
    Avg_Delivery_Hours=('delivery_time_hours', 'mean'),
    Top_Sender_Country=('sender_country',
        lambda x: x.mode()[0]))
```

Failed rows are grouped by **audit\_status** to size the operational risk; the "Passed" rows are grouped by **currency\_pair** to size real FX revenue per corridor. Each result exports straight to its own Excel file.

### Excel Report — Unresolved Transactions

| Final_Operational_Failure.xlsx (EXCEL REPORT PREVIEW) |            |                     |                   |                  |
|-------------------------------------------------------|------------|---------------------|-------------------|------------------|
| Audit Status                                          | Txn Volume | Total Impacted      | Fee at Risk       | Financial Impact |
| Missing Rate & Pair                                   | 193        | \$239,363.46        | \$4,280.87        | 5.01%            |
| Missing Status                                        | 143        | \$162,749.74        | \$2,813.24        | 3.41%            |
| Missing Financials                                    | 113        | \$94,326.00         | \$645.01          | 1.98%            |
| Missing Delivery Time                                 | 59         | \$74,522.98         | \$1,519.43        | 1.56%            |
| <b>Total</b>                                          | <b>508</b> | <b>\$570,962.18</b> | <b>\$9,258.55</b> | <b>11.96%</b>    |

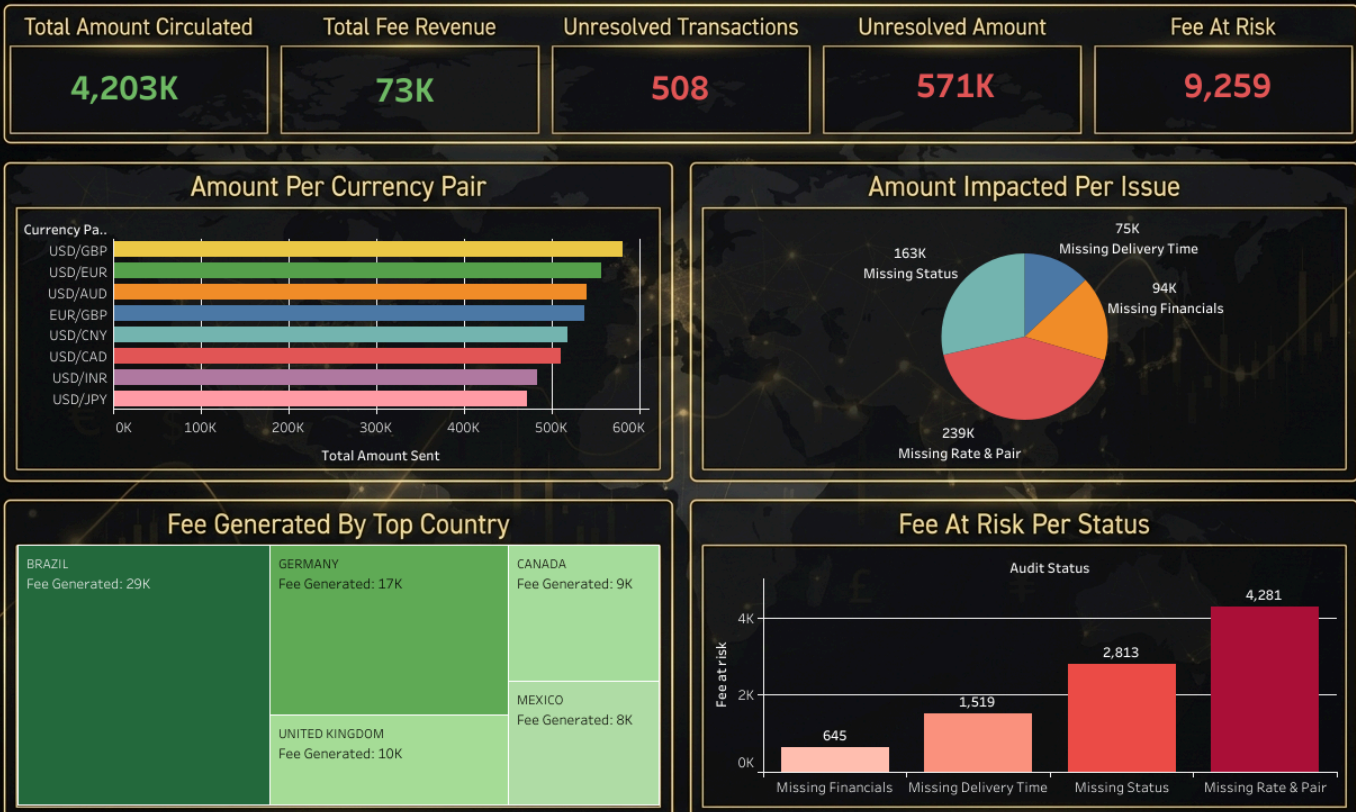
### Excel Report — FX Corridor Revenue (top rows)

| Final_FX_Corridor.xlsx (EXCEL REPORT PREVIEW) |                   |                     |              |                |              |
|-----------------------------------------------|-------------------|---------------------|--------------|----------------|--------------|
| Pair                                          | Fee Revenue       | Amount Sent         | Avg Hrs      | Top Sender     | Share        |
| USD/GBP                                       | \$10,599.43       | \$581,731.11        | 50.80        | Brazil         | 13.8%        |
| EUR/GBP                                       | \$9,537.11        | \$537,742.77        | 50.25        | United Kingdom | 12.8%        |
| USD/EUR                                       | \$9,453.07        | \$557,101.78        | 53.90        | Brazil         | 13.3%        |
| <b>USD/CNY</b>                                | <b>\$9,354.27</b> | <b>\$518,797.11</b> | <b>54.25</b> | <b>Canada</b>  | <b>12.3%</b> |

## STEP 5 THE DASHBOARD IN TABLEAU

Connected straight to the two Excel reports above — refreshing the source files updates every chart automatically.

### Foreign Exchange Revenue & Risk Analysis



**Pie & bar for risk** — the pie splits impacted amount by issue type, the bar ranks fee-at-risk per status, so the two views agree on where risk concentrates.

**Bar & treemap for revenue** — a horizontal bar ranks currency pairs by amount sent; a treemap sizes fee revenue by country without crowding an axis.

### Project File Structure

#### how the files connect

```
CSV/ fx_rates.csv international_payments.csv payment_status.csv & data_transfer2.py → loads the 3 csvs into MySQL (sql_analytics2) &
SQL_Analytics2.sql → audit checks, de-dup tables, v_payment_rates / v_delivery_status & python_analytics2.py → reads the views, cleans, flags,
builds both reports & Final_Operational_Failure.xlsx Final_FX_Corridor.xlsx & FX_Risk_Dashboard.twbx → Tableau workbook, connected to both
xlsx files
```

(STRUCTURE PREVIEW)